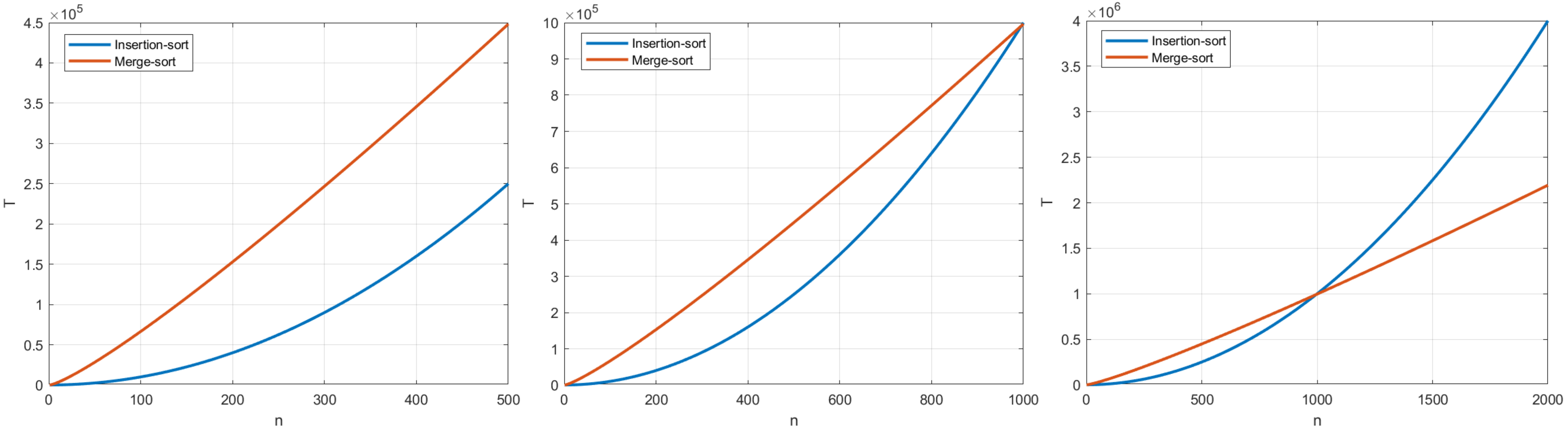
03. Growth of functions

Hyunjong Choi, Ph.D.
Assistant Professor of Computer Science
San Diego State University

Review

- ❑ **Insertion sort:** $T(n) = \Theta(n^2)$
- ❑ **Merge sort:** $T(n) = \Theta(n \lg n)$
- ❑ An extreme comparison:
  - $T_{Insertion}(n) = 1 \cdot n^2$, running on a fast computer
  - $T_{Merge}(n) = 100 \cdot n \lg n$, running on a slow computer

- The constant irrelevant in determining the running time
- Asymptotic efficiency



Asymptotic efficiency

- Asymptotic efficiency of algorithms:

The limit of $T(n)$ as the input size n increases to infinity

- It is the **order of growth** of $T(n)$ that matters
  - Discard lower-order terms
  - Ignore the coefficients of the highest-order term
  - Then, put the factor the highest-order term into Θ, O, Ω-notation

- An algorithm with better asymptotic efficiency will be preferred

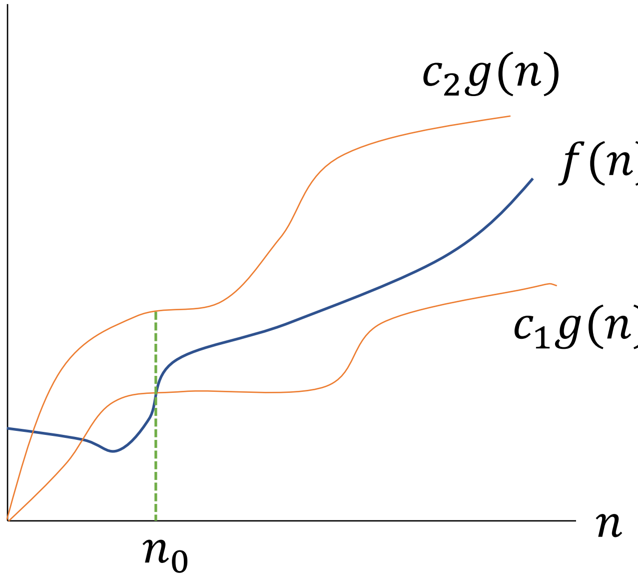
Overview of asymptotic notations

- O-notation
  - Characterizes an ***upper bound*** on the asymptotic behavior of a function.
  - e.g., $f(n) = 7n^3 + 100n^2 - 20n + 6$ → $\boldsymbol{O(n^3)}$
  - Then, can we write it as $\mathrm{O(n^4)}$ or $\mathrm{O(n^5)}$?
  - Generally, $\mathrm{O(n^c)}$ for any constant $\mathrm{c} \geq 3$.
- Ω-notation
  - Characterizes a ***lower bound*** on the asymptotic behavior of a function.
  - e.g., $f(n) = 7n^3 + 100n^2 - 20n + 6$ → $\boldsymbol{\Omega(n^3)}$
  - Generally, $\Omega(n^c)$ for any constant $\mathrm{c} \leq 3$.
- Θ-notation
  - Characterizes a tight bound on the asymptotic behavior of a function.
  - If we show $\mathrm{O}(f(n))$ and $\Omega(f(n))$, then the function is $\Theta(f(n))$.
  - e.g., $f(n) = 7n^3 + 100n^2 - 20n + 6,$ → $\boldsymbol{\Theta(n^3)}$

Θ-notation

- For a given function $g(n)$, $\Theta(g(n))$ is the set of functions

$$\Theta\big(g(n)\big) = \{f(n)\text{: there exist positive constants } c_1, c_2, \text{and } n_0 \text{ such that } 0 \le c_1 g(n) \le f(n) \le c_2 g(n) \text{ for all } n \ge n_0\}$$



→ $g(n)$ is an asymptotically **tight bound** for $f(n)$

→ It requires that $f(n)$ and $g(n)$ **asymptotically nonnegative**

- Technically, it should be "$f(n) \in \Theta(g(n))$", but written as $f(n) = \Theta(g(n))$

Representation of Θ-notation

- Ignore lower-order terms and coefficient of the highest order of term
  - Example: $f(n) = \frac{1}{2}n^2 - 3n = \Theta(n^2)$ → Why?
    - ✓ Need to determine positive constants c_1, c_2, and n_0 such that

$$c_1 n^2 \leq \frac{1}{2}n^2 - 3n \leq c_2 n^2, \text{ for all } n \geq n_0$$

    - ✓ Dividing by n^2 yields

$$c_1 \leq \frac{1}{2} - \frac{3}{n} \leq c_2$$

If we choose any constant $c_2 \geq \frac{1}{2}$, the right inequality always holds

Therefore, by choosing $c_1 = \frac{1}{14}$, $c_2 = \frac{1}{2}$, and $n_0 = 7$, $\boldsymbol{\frac{1}{2}n^2 - 3n = \Theta(n^2)}$ **holds**

When $n \geq 7$, $\frac{1}{2} - \frac{3}{n} \geq \frac{1}{14}$, thus for any constant $c_1 \leq \frac{1}{14}$, the left inequality always holds

Representation of Θ-notation (cont.)

- There are other choices for c_1, c_2, and n_0
- But the important thing is that some choices exists.
- These constants depend on the coefficients of the function, $\frac{1}{2}n^2 - 3n$

Representation of Θ-notation (cont.)

- Example: To verify that $6n^3 \neq \Theta(n^2)$
  - Suppose that c_2 and n_0 exist such that $6n^3 \leq c_2 n^2$ holds for all $n \geq n_0$
  - Dividing by n^2 yields:

$$n \leq \frac{c_2}{6}$$

  - This is a contradiction because n can be arbitrarily large.
  - **Therefore, $\boldsymbol{6n^3 \neq \Theta(n^2)}$**

< Proof by contradiction >

When we try to prove the statement is true.

Step 1. Assume that the statement is not true.

Step 2. Show that the result is wrong.

Step 3. Therefore, the initial statement is true.

Representation of Θ-notation (cont.)

- Generic quadratic function
  - Example: $f(n) = n^2 + 2n + 1$
    - ✓ To show $f(n) = \Theta(n^2)$, we need to find c_1, c_2, and n_0 such that

$$c_1 n^2 \leq n^2 + 2n + 1 \leq c_2 n^2, \text{ for all } n \geq n_0$$

    - ✓ The inequalities become:

$$0 \leq (1 - c_1)n^2 + 2n + 1 \text{ and } (1 - c_2)n^2 + 2n + 1 \leq 0$$

    - ✓ Thus, we need to have: $c_1 < 1 < c_2$
    - ✓ And that n be greater than the larger root of the two equations:

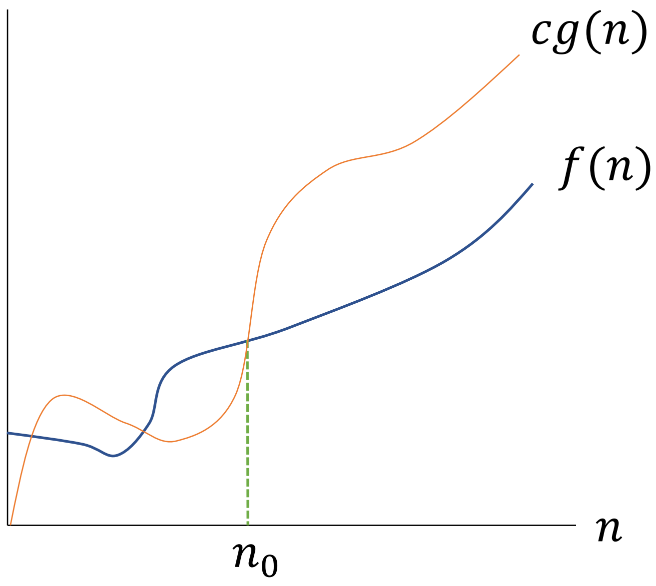
$$\begin{cases} (1 - c_2)n^2 + 2n + 1 = 0 \\ (1 - c_1)n^2 + 2n + 1 \ = 0 \end{cases}$$

    - ✓ c_1 and c_2 can have multiple possible values, as long as $c_1 < 1 < c_2$
    - ✓ If $c_1 = \frac{1}{4}$ and $c_2 = \frac{9}{4}$, then $n_0 = 2$

O-notation

- For a given function $g(n)$, $O(g(n))$ is the set of functions:

$$O\big(g(n)\big) = \{f(n)\text{: there exist positive constants } c, \text{and } n_0 \text{ such that } 0 \le f(n) \le cg(n) \text{ for all } n \ge n_0\}$$

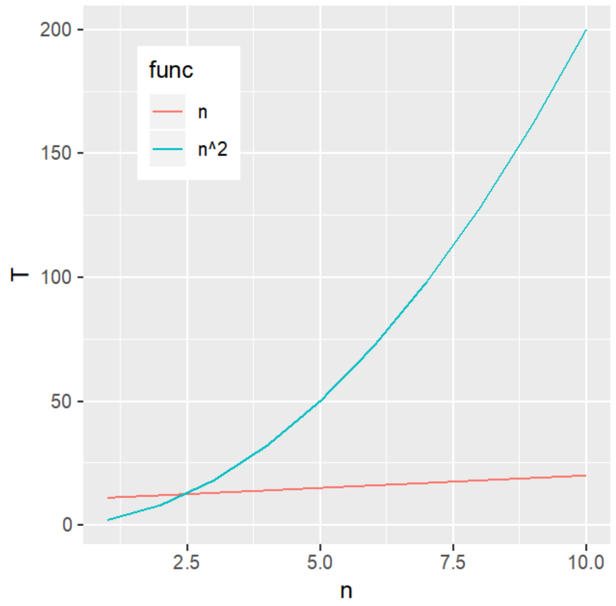


✓ To the right of n_0, $f(n)$ is always below $cg(n)$

Note that $\boldsymbol{f(n) = \Theta(g(n))}$ ***implies*** $\boldsymbol{f(n) = O(g(n))}$

Tightness of the bound

- ❑ Θ is tight; O is loose
- ❑ Tightness of O notation
  - By $f(n) = O(g(n))$, it only means that some constant multiple of $g(n)$ is an asymptotic upper bound of $f(n)$, with no claim about *how tight* an upper bound it is.

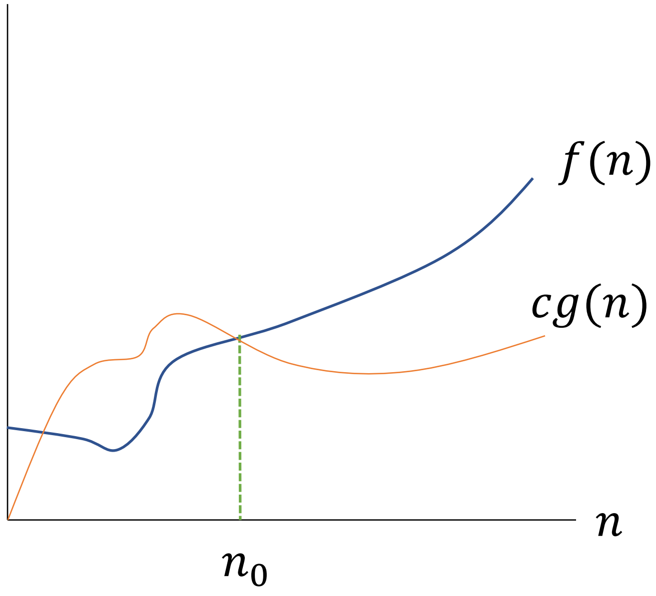


✓ n^2 is definitely an upper bound of n, but it is not tight.

Ω-notation

- For a given function $g(n)$, $\Omega(g(n))$ is the set of functions

$$\Omega(g(n)) = \{f(n)\text{: there exist positive constants } c, \text{and } n_0 \text{ such that } 0 \le cg(n) \le f(n) \text{ for all } n \ge n_0\}$$



✓ To the right of n_0, $f(n)$ is always **above** $cg(n)$

No matter what particular input of size n is chosen, the running time on that input is *at least* a constant times $g(n)$.

Relationship between Θ-, O-, and Ω-notation

< Theorem >

For any two functions $f(n)$ and $g(n)$, we have $f(n) = \Theta(g(n))$ ***if and only if*** $f(n) = O(g(n))$ **and** $f(n) = \Omega(g(n))$.

Revisit insertion sort

INSERTION-SORT(A)	Cost	Number of times
for j = 2 **to** A.length	c_1	n
key = A[j]	c_2	$n-1$
// Insert A[j] into the sorted sequence A[1 .. j-1]	0	$n-1$
i = j – 1	c_4	$n-1$
while i > 0 and A[i] > key	c_5	$\sum_{j=2}^{n} t_j$
A[i + 1] = A[i]	c_6	$\sum_{j=2}^{n} (t_j - 1)$
i = i – 1	c_7	$\sum_{j=2}^{n} (t_j - 1)$
A[i + 1] = key	c_8	$n-1$

$$T(n) = c_1 n + c_2(n-1) + c_4(n-1) + c_5 \sum_{j=2}^{n} t_j + c_6 \sum_{j=2}^{n} (t_j - 1) + c_7 \sum_{j=2}^{n} (t_j - 1) + c_8(n-1)$$

Best-case: $t_j = 1, \quad T(n) = an + b \quad = \Theta(n)$

Worst-case: $t_j = j, \quad T(n) = an^2 + bn + c \quad = \Theta(n^2)$

$$\boxed{\begin{array}{c} O(n^2) \\ \uparrow \\ T(n) \\ \downarrow \\ \Omega(n) \end{array}}$$

- The lower bound is **not** $\Omega(n^2)$ because there exists an input (already sorted) for which the running time is $\Theta(n)$

Optional

o-notation (small-oh)

- The bound $2n^2 = \Theta(n^2)$ is asymptotically tight, but $2n = O(n^2)$ is not.
- It denotes an *upper bound that is not asymptotically tight*

$$o(g(n)) = \{f(n)\text{: for } \boldsymbol{any} \text{ positive constants } c > 0, \text{there exists a constant } n_0 \text{ such that } 0 \le f(n) < cg(n) \text{ for all } n \ge n_0\}$$

  - ✓ For example, $2n = o(n^2)$, but $2n^2 \neq o(n^2)$.

- Difference from O-notation:
  - If $f(n) = O(g(n))$, the bound $0 \le f(n) \le cg(n)$ holds for **some** constant $c > 0$.
  - If $f(n) = o(g(n))$, the bound $0 \le f(n) < cg(n)$ holds for **all** constant $c > 0$.

- $f(n)$ becomes insignificant relative to $g(n)$ as n approaches infinity, i.e., $\lim_{n\to\infty} \frac{f(n)}{g(n)} = 0$

Optional

ω-notation (small-omega)

- ω-notation is to Ω-notation as o-notation is to O-notation.
- It denotes a lower bound that is *not* asymptotically tight.

$$\omega(g(n)) = \{f(n): \text{ for } \textbf{\textit{any}} \text{ positive constants } c > 0, \text{there exists a constant } n_0 \text{ such that } 0 \leq cg(n) < f(n) \text{ for all } n \geq n_0\}$$

- For example, $\frac{n^2}{2} = \omega(n)$, but $\frac{n^2}{2} \neq \omega(n^2)$

- $f(n) = \omega(g(n))$ if and only if $g(n) = o(f(n))$

- $f(n)$ becomes arbitrarily large relative to $g(n)$ as n approaches infinity, i.e., $\lim_{n\to\infty} \frac{f(n)}{g(n)} = \infty$

Asymptotic notation in equations

- Asymptotic notations can denote *anonymous functions* that we do not care to name
  - The worst-case running time of merge sort, $T(n) = 2T\left(\frac{n}{2}\right) + \Theta(n)$
  - It means that $T(n) = 2T\left(\frac{n}{2}\right) + f(n)$, and $f(n)$ is in the set $\Theta(n)$.
- Asymptotic notations on both sides:
  - $2n^2 + \Theta(n) = \Theta(n^2)$
  - No matter how the left side is chosen, there is a way to choose the anonymous function on the right to make the equation valid.
- Chaining equations
  - $2n^2 + 3n + 1 = 2n^2 + \Theta(n) = \Theta(n^2)$

Relational properties of asymptotic notations

- Transitivity:

$$f(n) = \Theta(g(n)) \text{ and } g(n) = \Theta(h(n)) \quad \text{imply} \quad f(n) = \Theta(h(n))$$
$$f(n) = O(g(n)) \text{ and } g(n) = O(h(n)) \quad \text{imply} \quad f(n) = O(h(n))$$

Same applies to Ω-, o-, and ω-notations

- Analogy to comparing real numbers:

$f(n) = \Theta(g(n))$ is like $a = b$
$f(n) = O(g(n))$ is like $a \leq b$
$f(n) = \Omega(g(n))$ is like $a \geq b$
$f(n) = o(g(n))$ is like $a < b$
$f(n) = \omega(g(n))$ is like $a > b$

Relational properties (cont.)

- Reflexivity:

$$f(n) = \Theta(f(n))$$
$$f(n) = O(f(n))$$
$$f(n) = \Omega(f(n))$$

- Symmetry:

$f(n) = \Theta(g(n))$ if and only $g(n) = \Theta(f(n))$

- Transpose symmetry:

$f(n) = O(g(n))$ if and only $g(n) = \Omega(f(n))$

$f(n) = o(g(n))$ if and only $g(n) = \omega(f(n))$

Asymptotic bounds for common functions

- Any *exponential* function with a base greater than 1 grows faster than any *polynomial* function

$$n^b = o(a^n), a > 1 \qquad \lim_{n\to\infty} \frac{n^b}{a^n} = 0 \qquad (1)$$

- Any positive polynomial function grows faster than any *polylogarithmic* function
  - Polylogarithmic function: $f(n) = (\lg n)^k = \lg^k n$

Replace $\lg n$ for n and 2^a for a in equation (1), yielding:

$$\lim_{n\to\infty} \frac{\lg^b n}{(2^a)^{\lg n}} = \lim_{n\to\infty} \frac{\lg^b n}{n^a} = 0 \quad \Rightarrow \quad \lg^b n = o(n^a)$$

Asymptotic bounds for common functions (cont.)

- Factorials $$n! = \begin{cases} 1, & \text{if } n = 0 \\ n \cdot (n-1), & \text{if } n > 0 \end{cases}$$

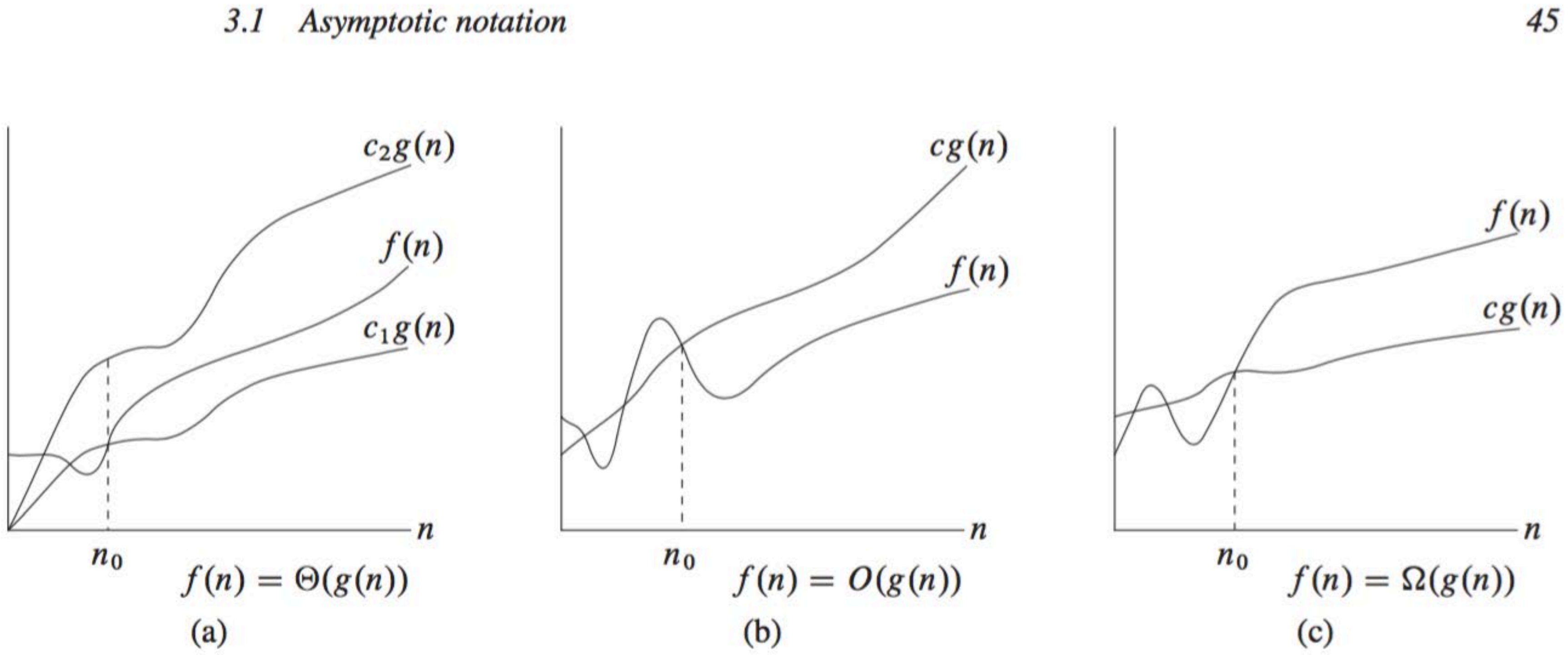
- A weak upper bound: $n! = o(n^n)$

- We can prove:

$$n! = \omega(2^n)$$

$$\lg(n!) = \Theta(n \cdot \lg n)$$

Recap



Reading requirement: Textbook 3rd edition, page 43 – 53

Recap

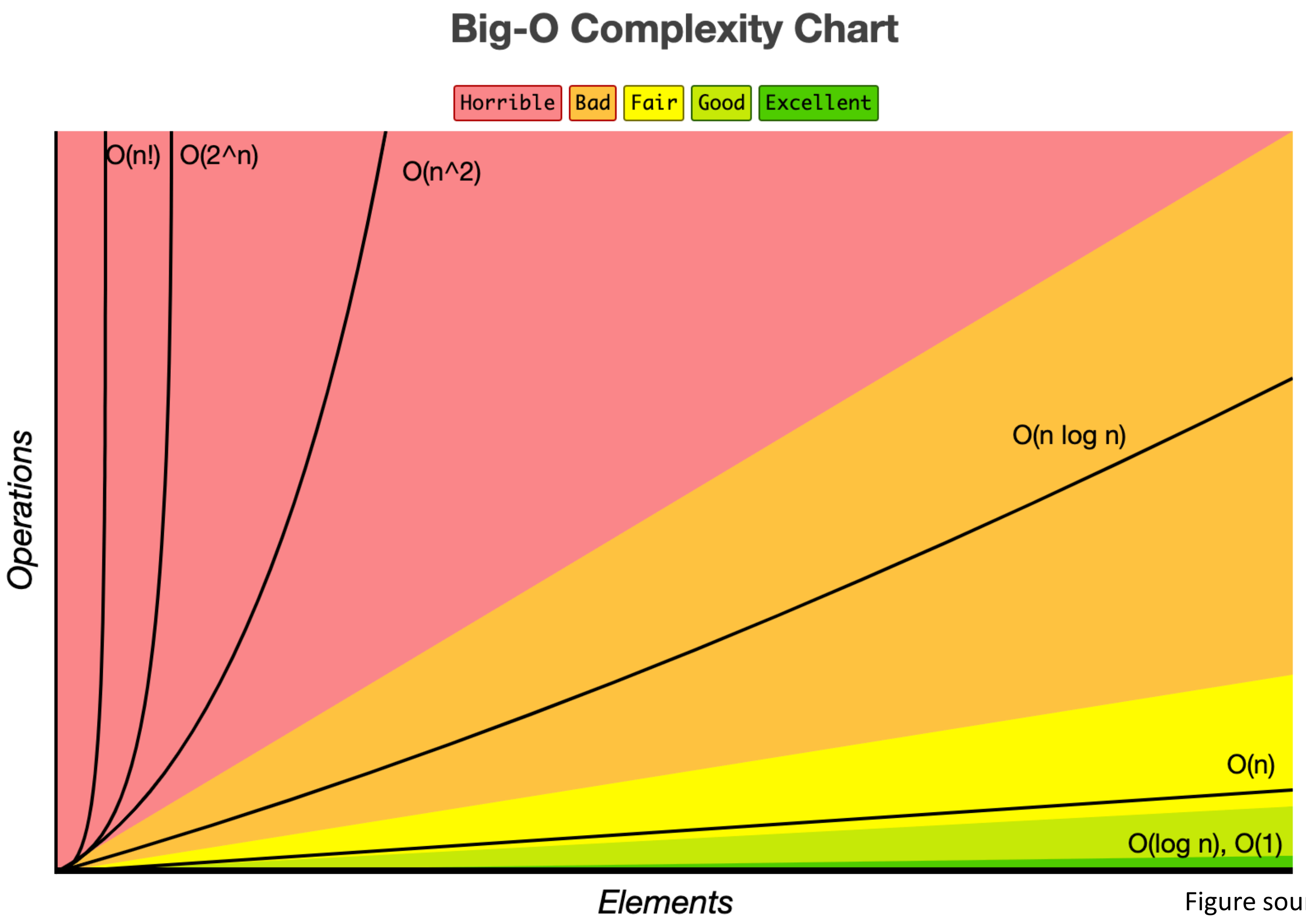


Figure source: https://www.bigocheatsheet.com/

Example

Problem

- True or False?
  - Q1. Insertion sort's ***worst-case*** running time is $O(n^2), \Omega(n^2)$, and $\Theta(n^2)$?
    - ✓ The worst-case running time (n^2) grows **no faster than** the rate of growth of function n^2.
    - ✓ The worst-case running time (n^2) grows **at least** the rate of growth of function n^2.
    - ✓ The worst-case running time (n^2) grows **precisely** at the rate of growth of function n^2.
  - Q2-1) Insertion sort's running time is $\Theta(n^2)$?
    - ✓ Insertion sort's running time grows precisely at the rate of growth of function n^2.
  - Q2-2) How about "Insertion sort's running time is $O(n^2)$"?
    - ✓ Insertion sort's running time grows no faster than the rate of growth of function n^2.

Q & A